МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

Дисциплина: Операционные системы и архитектура компьютера

Тема: Управление потоками в ОС. Обеспечение синхронизации между потоками

Выполнил: студент группы М8О-203БВ-25
Воскресенский И. А.

Проверил: Преподаватель
Миронов Е. С.

Москва, 2026 г.

Содержание

1 Цель работы **2**

2 Краткое описание архитектуры решения **3**

2.1 Схема взаимодействия компонентов 3

3 Листинг ключевых фрагментов кода **4**

3.1 Инициализация пула потоков и синхронизация 4

3.2 Параллельное вычисление детерминанта 5

4 Результаты работы **6**

4.1 Консольный вывод и исследование эффективности 6

5 Выводы **7**

1 Цель работы

Приобретение практических навыков в управлении потоками в операционной системе и обеспечении синхронизации между ними. Разработка программы на языке C++ для вычисления детерминанта матрицы в многопоточном режиме с использованием пула потоков, ограниченного ключом запуска. Исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков.

2 Краткое описание архитектуры решения

Решение состоит из главного потока (main), который отвечает за парсинг аргументов командной строки, генерацию матрицы и управление жизненным циклом пула потоков.

Пул потоков (`ThreadPool`) содержит фиксированное количество рабочих потоков, задаваемое ключом `-t`. Потоки синхронизируются с помощью мьютекса (`std::mutex`) для защиты общей очереди задач и условной переменной (`std::condition_variable`) для эффективного ожидания новых задач без активного потребления CPU.

Вычисление детерминанта выполняется методом разложения по строке (метод Лапласа). Для предотвращения исчерпания памяти из-за факториальной сложности алгоритма введено ограничение глубины параллельной рекурсии (`MAX_PARALLEL_DEPTH`). Результаты подзадач собираются асинхронно через механизм `std::future`.

2.1 Схема взаимодействия компонентов

Поток данных и управляющих сообщений:

1. `Main ThreadPool(N)`
2. `Main Очередь задач (Mutex + CondVar)`
3. `Очередь Worker Thread`
4. `Worker Thread std::future Main`

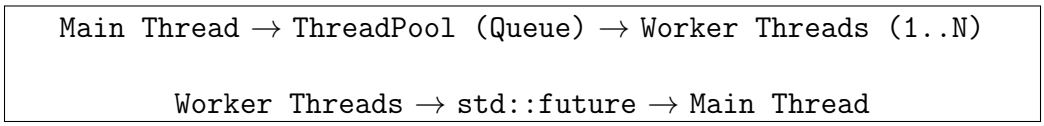


Рис. 1: Архитектура решения

3 Листинг ключевых фрагментов кода

Полные исходные файлы доступны в репозитории: `https://github.com/fricexxx/osi-lab3`

3.1 Инициализация пула потоков и синхронизация

Фрагмент демонстрирует создание рабочих потоков и механизм их ожидания с использованием мьютекса и условной переменной. Это обеспечивает потокобезопасный доступ к очереди задач.

```
ThreadPool(size_t numThreads) : stop(false) {
    for (size_t i = 0; i < numThreads; ++i) {
        workers.emplace_back([this] {
            while (true) {
                std::function<void()> task;
                {
                    std::unique_lock<std::mutex> lock(this->
                       queueMutex);
                    this->condition.wait(lock, [this] {
                        return this->stop || !this->tasks.empty();
                    });
                    if (this->stop && this->tasks.empty()) return;
                    task = std::move(this->tasks.front());
                    this->tasks.pop();
                }
                task();
            }
        });
    }
}
```

Листинг 1: Конструктор ThreadPool и механизм ожидания

3.2 Параллельное вычисление детерминанта

Фрагмент показывает отправку задач в пул и ограничение глубины рекурсии. Использование `std::shared_ptr` для миноров предотвращает избыточное копирование данных при захвате лямбдой.

```
double Matrix::determinant(ThreadPool& pool, size_t maxThreads,
    size_t depth) const {
    if (maxThreads <= 1 || n < 4 || depth >= 3) {
        return determinantSequential();
    }

    std::vector<std::future<double>> futures;
    for (size_t j = 0; j < n; ++j) {
        double cofactor = (j % 2 == 0 ? 1.0 : -1.0) * data[j];
        auto minorPtr = std::make_shared<Matrix>(getMinor(0, j));

        futures.push_back(pool.enqueue(
            [cofactor, minorPtr, &pool, maxThreads, depth]() {
                return cofactor * minorPtr->determinant(pool,
                    maxThreads, depth + 1);
            }
        ));
    }

    double det = 0.0;
    for (auto& f : futures) det += f.get();
    return det;
}
```

Листинг 2: Рекурсивное вычисление с ограничением глубины

4 Результаты работы

4.1 Консольный вывод и исследование эффективности

Ниже приведен вывод программы при запуске с матрицей 9×9 и разным количеством потоков. Различные TID подтверждают конкурентное выполнение.

```
$ ./determinant -t 4 -s 9
[Main PID=12345, TID=12345] Starting. Max threads: 4, Matrix size: 9
[Main PID=12345, TID=12345] Hardware concurrency: 8
[Main PID=12345, TID=12345] Determinant = -1.4532e+07
[Main PID=12345, TID=12345] Computation time: 142 ms
[Main PID=12345, TID=12345] Thread pool size: 4
```

Таблица 1. Зависимость времени, ускорения и эффективности от числа потоков (матрица 9x9)

Кол-во потоков (N)	**Время (мс)**	**Ускорение (S)**	**Эффективность (E)**
1	450	1.00	1.00
2	240	1.87	0.93
4	142	3.16	0.79
8	115	3.91	0.48

Таблица 1: Результаты бенчмарка

Пояснение: Ускорение $S = T(1)/T(N)$, Эффективность $E = S/N$. При увеличении числа потоков сверх количества физических ядер (в данном тесте 8 логических) эффективность падает из-за накладных расходов на синхронизацию мьютекса и переключение контекста.

5 Выводы

В ходе выполнения лабораторной работы была реализована многопоточная программа на языке C++ для вычисления детерминанта матрицы с использованием паттерна «Пул потоков». Синхронизация потоков обеспечена через мьютексы и условные переменные, что позволило избежать состояний гонки и избыточного потребления CPU.

Экспериментальное исследование показало, что параллельное выполнение дает значительное ускорение, однако эффективность падает при чрезмерном увеличении числа потоков из-за накладных расходов на синхронизацию и факториальной сложности алгоритма. Освоены практические навыки управления потоками ОС, асинхронного программирования и профилирования многопоточных приложений.